

SGit vaults in 90 seconds

Encrypted, versioned folders the server can never read

2026-09-01

made by a Claude Code agent, unattended

13 scenes · 02:05 landscape

source: reel.json

00:00

OPENING 9.9 S

What this video covers

In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

00:09

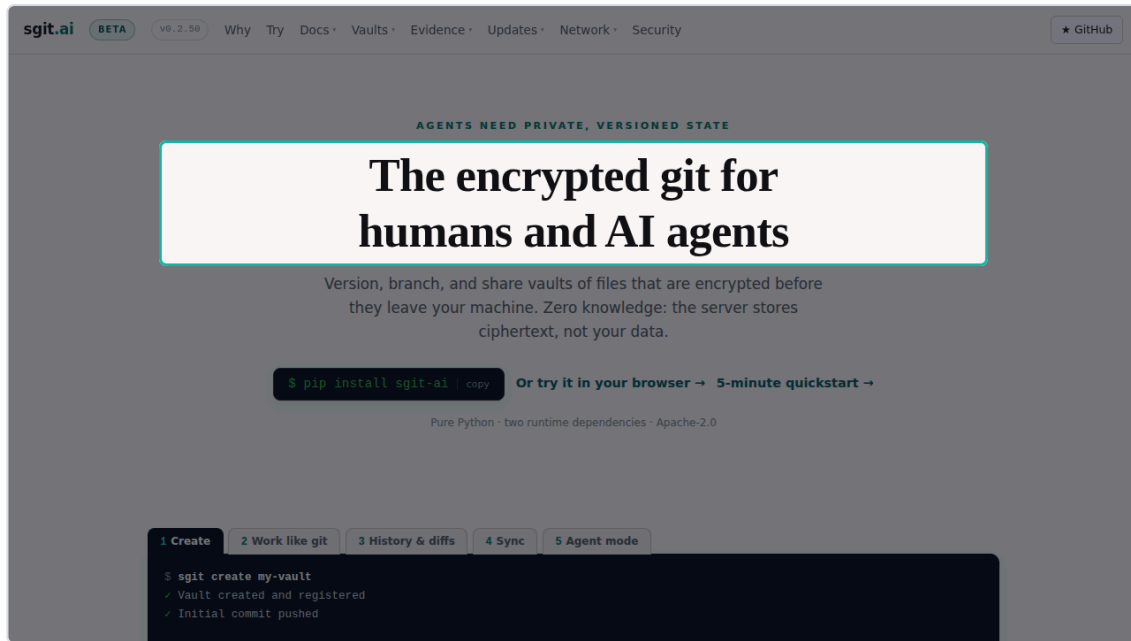
SCENE 1 OF 13 S01 7.9 S

Encrypted before it leaves

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

HOW THIS IMAGE WAS CREATED

- Opened <https://sgit.ai/>
- Spotlit the heading “The encrypted git”



00:17

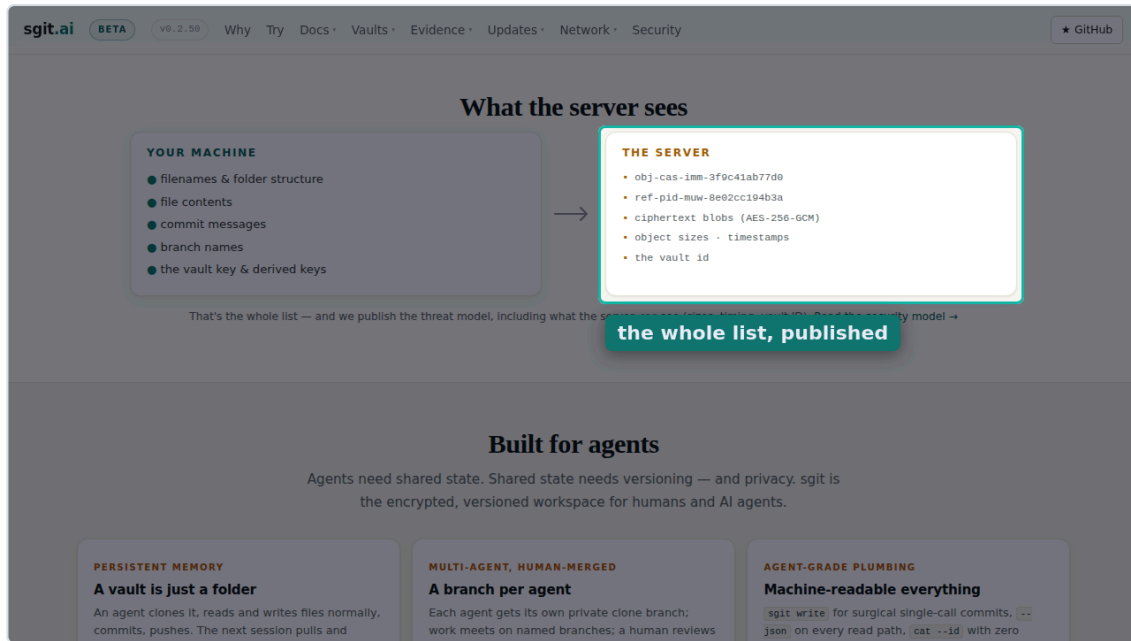
SCENE 2 OF 13 S02 9.2 S

The server sees ciphertext

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

HOW THIS IMAGE WAS CREATED

- ▶ Opened `https://sgit.ai/`
- ▶ Scrolled to “What the server sees”
- ▶ Spotlighted the “The server” column
- ▶ Added the label “the whole list, published”



00:27

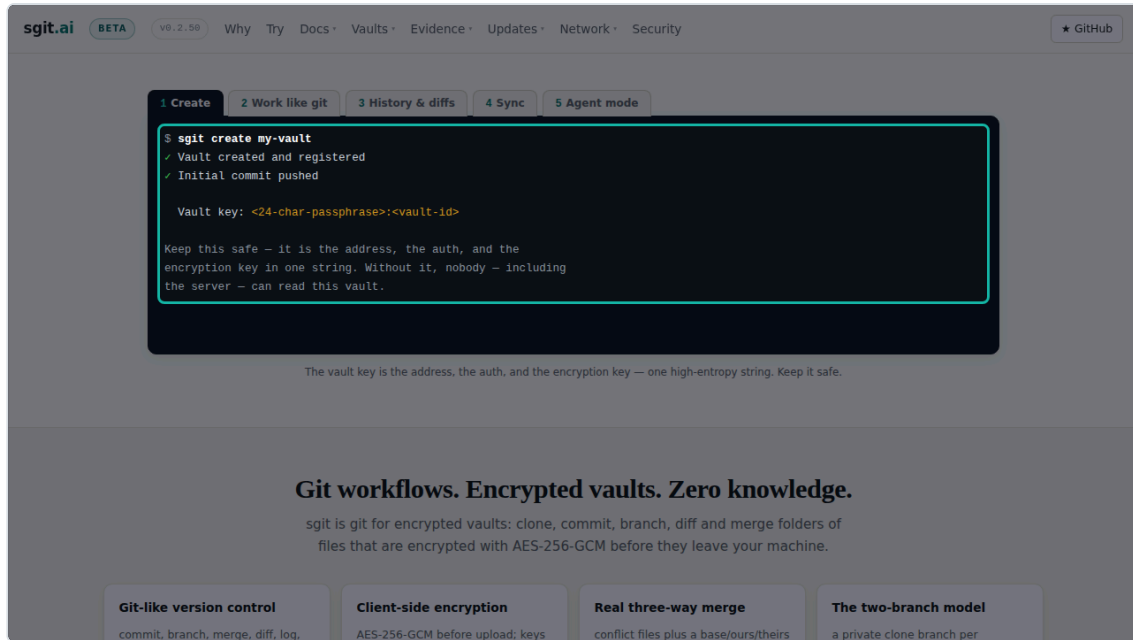
SCENE 3 OF 13 S03 2.9 S

One command

Creating a vault is one command.

HOW THIS IMAGE WAS CREATED

- ▶ Opened <https://sgit.ai/>
- ▶ Scrolled to “Work like git”
- ▶ Spotlighted the terminal block



00:29

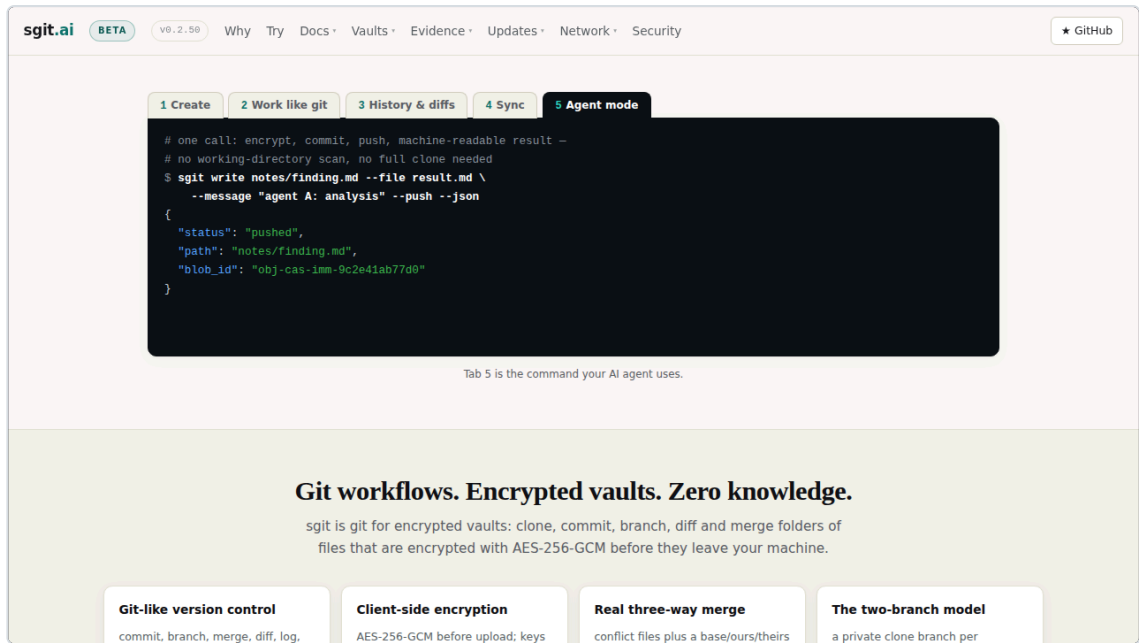
SCENE 4 OF 13 S04 7.6 S

Work like git

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

HOW THIS IMAGE WAS CREATED

- ▶ Recorded a clip of <https://sgit.ai/>
- ▶ Scrolled to “Work like git”
- ▶ Clicked through the tabs: Work like git, History & diffs, Sync, Agent mode
- ▶ Used the last frame as the still

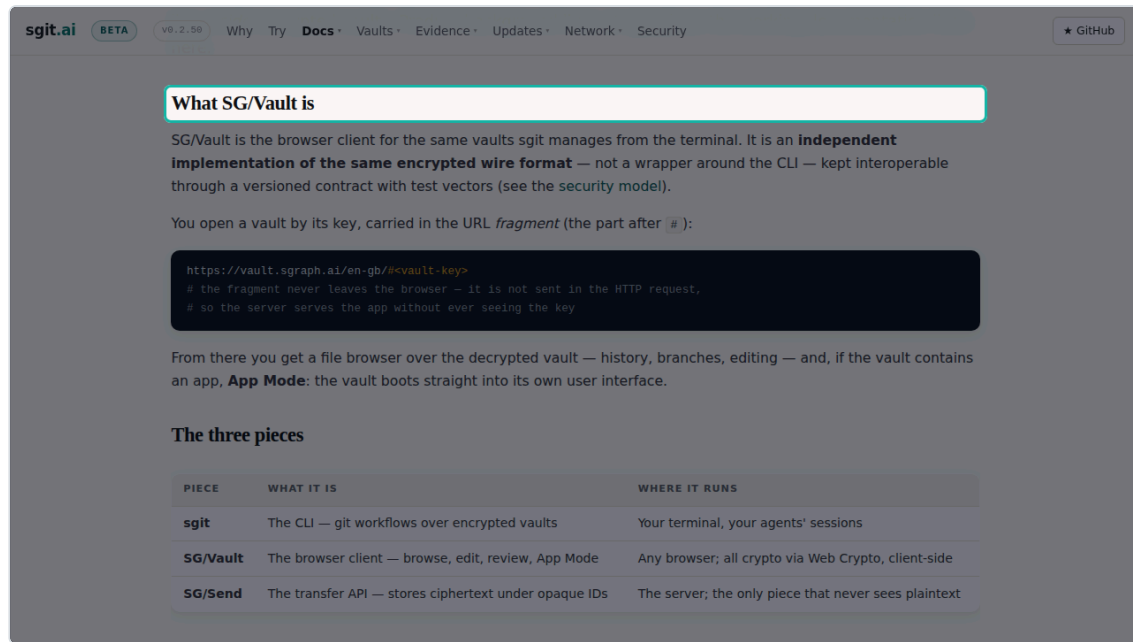


Same vault, in the browser

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

HOW THIS IMAGE WAS CREATED

- ▶ Opened `https://sgit.ai/vault/`
- ▶ Scrolled to “What SG/Vault is”
- ▶ Spotlighted the heading “What SG/Vault is”



00:46

SCENE 6 OF 13 S06 9.5 S

The key never leaves the browser

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

HOW THIS IMAGE WAS CREATED

- ▶ Opened `https://sgit.ai/vault/`
- ▶ Scrolled to “You open a vault by its key”
- ▶ Spotlit the code block after the heading
- ▶ Added the label “the key lives after the #”

The screenshot shows the sgit.ai website with a navigation bar at the top containing links like 'Docs', 'Vaults', 'Evidence', 'Updates', 'Network', and 'Security'. The main content area explains how to open a vault by its key in the URL fragment. A code block is highlighted with a red border, containing the following text:

```
https://vault.sgraph.ai/en-gb/#<vault-key>
# the fragment never leaves the browser — it is not sent in the HTTP request,
# so the server serves the app without ever seeing the key
```

Below the code block, a red label with white text reads "the key lives after the #". The text continues to explain that the fragment is used to open a decrypted vault and its user interface.

The three pieces

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

Vault apps: the app is the vault

A vault can contain an `index.html` plus an `app.json`. When the vault opens, the app launches — full-screen, talking to the vault through a runtime bridge, with the encrypted blob serving as both the storage and the distribution mechanism. Sharing the vault link *is* deploying the app.

00:55

SCENE 7 OF 13 S07 12.9 S

Three pieces, one wire format

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

HOW THIS IMAGE WAS CREATED

- Opened `https://sgit.ai/vault/`
- Scrolled to “The three pieces”
- Spotlighted the table after the heading

The screenshot shows the sgit.ai website with a navigation bar at the top containing links like Docs, Vaults, Evidence, Updates, Network, and Security. A GitHub link is also present. The main content area is titled 'The three pieces' and features a table with three columns: PIECE, WHAT IT IS, and WHERE IT RUNS. The table lists three components: sgit (CLI), SG/Vault (browser client), and SG/Send (transfer API). Below the table, there is a section titled 'Vault apps: the app is the vault' which explains that a vault can contain an index.html and an app.json, and that the app launches full-screen when the vault opens. A callout box highlights that the entire website is a vault app generated by sgit push. At the bottom, there is a section titled 'Three on-ramps' with a table that is partially visible.

PIECE	WHAT IT IS	WHERE IT RUNS
sgit	The CLI — git workflows over encrypted vaults	Your terminal, your agents' sessions
SG/Vault	The browser client — browse, edit, review, App Mode	Any browser; all crypto via Web Crypto, client-side
SG/Send	The transfer API — stores ciphertext under opaque IDs	The server; the only piece that never sees plaintext

Vault apps: the app is the vault

A vault can contain an `index.html` plus an `app.json`. When the vault opens, the app launches — full-screen, talking to the vault through a runtime bridge, with the encrypted blob serving as both the storage and the distribution mechanism. Sharing the vault link *is* deploying the app.

You are looking at one. This entire website is a vault app: generated pages, shared assets loaded through the bridge, deployed by `sgit push`. The admin section documents its engineering and doubles as the reference implementation.

Three on-ramps

YOU WANT TO...	USE	CODE NEEDED
Public documents, collectibles, hub pages	Meddows	In the browser, client-side

01:08

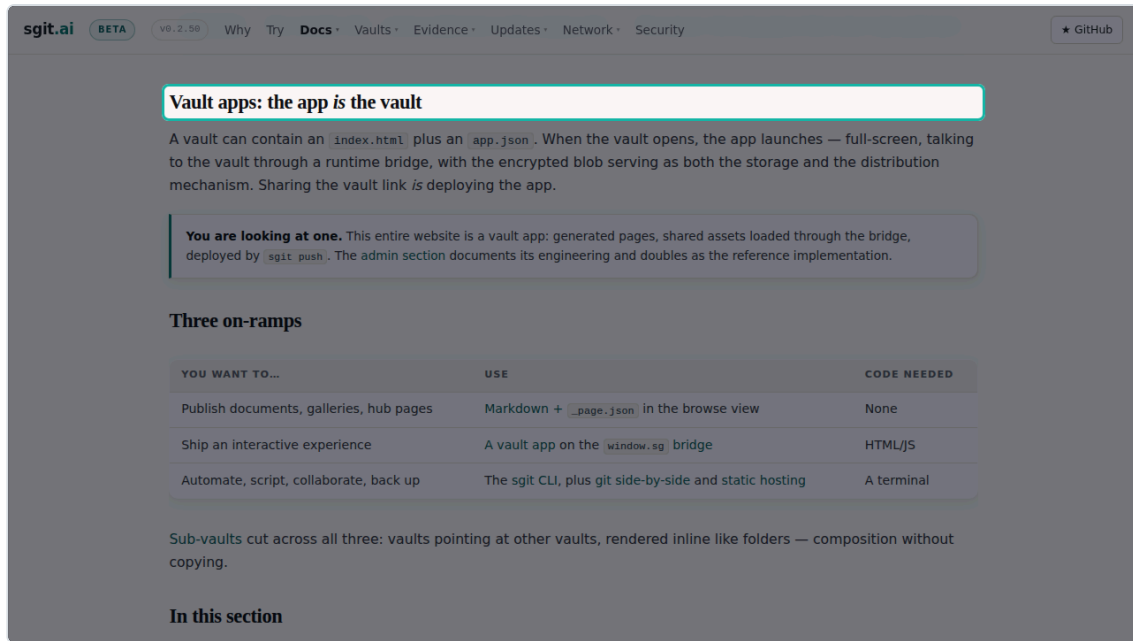
SCENE 8 OF 13 S08 5.7 S

The app is the vault

A vault can contain an app. Sharing the vault link is deploying the app.

HOW THIS IMAGE WAS CREATED

- ▶ Opened `https://sgit.ai/vault/`
- ▶ Scrolled to “Vault apps: the app is the vault”
- ▶ Spotlighted the heading “Vault apps: the app is the vault”

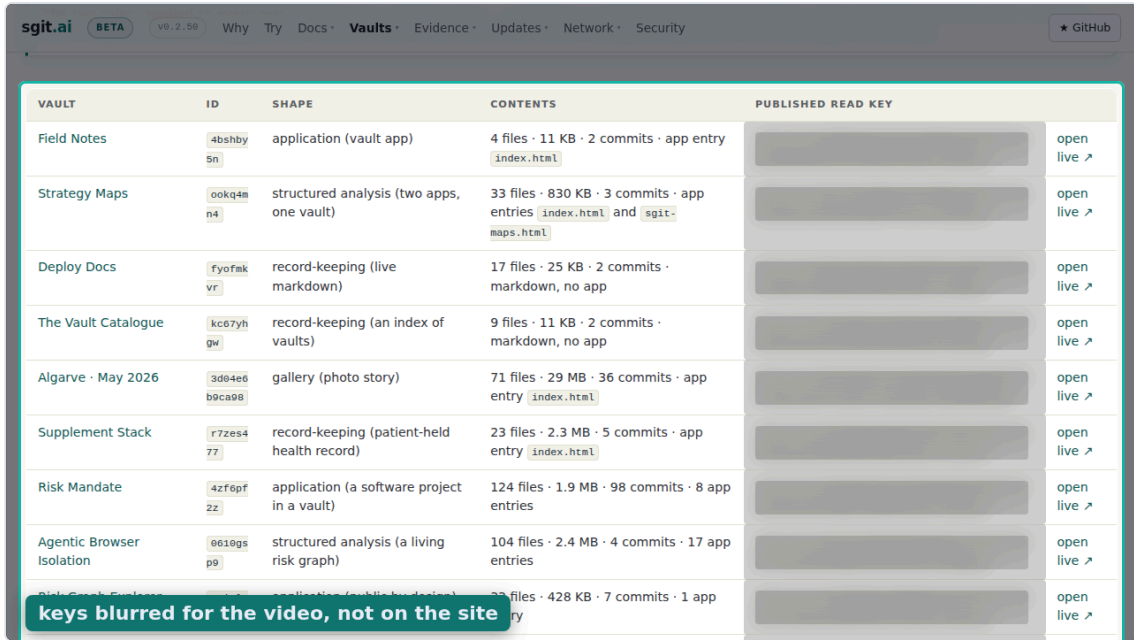


Read keys, published on purpose

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

HOW THIS IMAGE WAS CREATED

- Opened `https://sgit.ai/demos/vaults/index.html`
- Scrolled to “Published read key”
- Spotlighted the table after the heading
- Blurred the read-key cells
- Added the label “keys blurred for the video, not on the site”



sgit.ai BETA v9.2.50 Why Try Docs Vaults Evidence Updates Network Security GitHub

VAULT	ID	SHAPE	CONTENTS	PUBLISHED READ KEY	
Field Notes	4bshby 5n	application (vault app)	4 files · 11 KB · 2 commits · app entry index.html		open live ↗
Strategy Maps	ookq4m n4	structured analysis (two apps, one vault)	33 files · 830 KB · 3 commits · app entries index.html and sgit-maps.html		open live ↗
Deploy Docs	fyofmk vr	record-keeping (live markdown)	17 files · 25 KB · 2 commits · markdown, no app		open live ↗
The Vault Catalogue	kce7yh gw	record-keeping (an index of vaults)	9 files · 11 KB · 2 commits · markdown, no app		open live ↗
Algarve · May 2026	3d04e0 b9ca98	gallery (photo story)	71 files · 29 MB · 36 commits · app entry index.html		open live ↗
Supplement Stack	r7zes4 77	record-keeping (patient-held health record)	23 files · 2.3 MB · 5 commits · app entry index.html		open live ↗
Risk Mandate	4zf6pf 2z	application (a software project in a vault)	124 files · 1.9 MB · 98 commits · 8 app entries		open live ↗
Agentic Browser Isolation	0610gs p9	structured analysis (a living risk graph)	104 files · 2.4 MB · 4 commits · 17 app entries		open live ↗
Risk Graph Explorer		application (publicly hosted)	22 files · 428 KB · 7 commits · 1 app entry		open live ↗

keys blurred for the video, not on the site

01:23

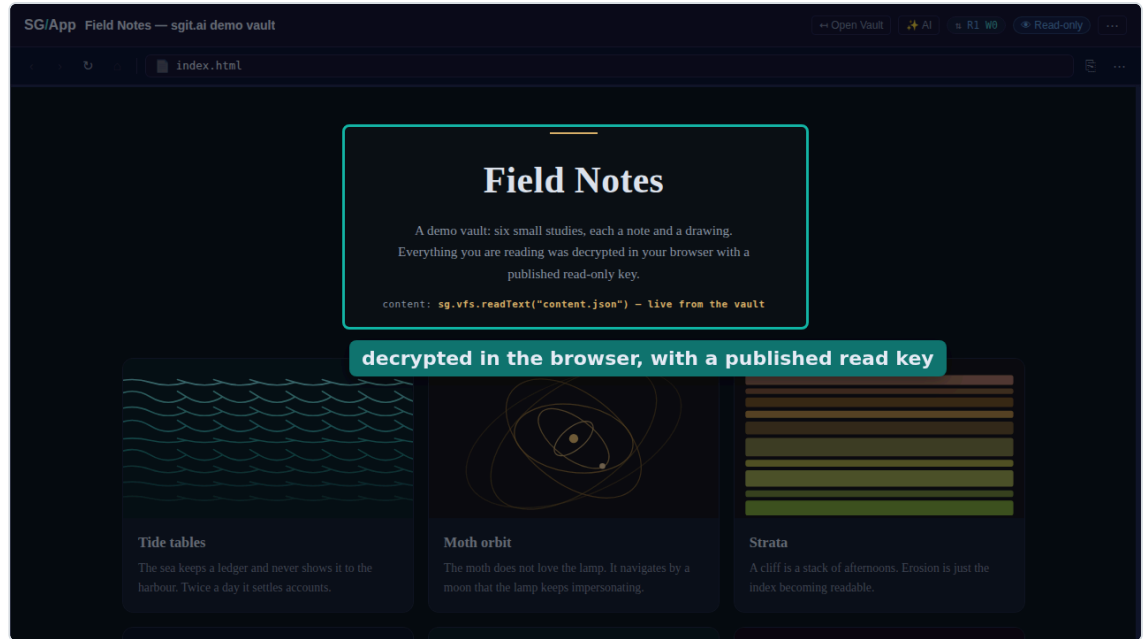
SCENE 10 OF 13 S10 9.1 S

Decrypted in your browser

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

HOW THIS IMAGE WAS CREATED

- ▶ Opened the published vault “Field Notes”, opened live in SG/App
- ▶ Spotlitged a hand-placed rectangle
- ▶ Added the label “decrypted in the browser, with a published read key”



01:32

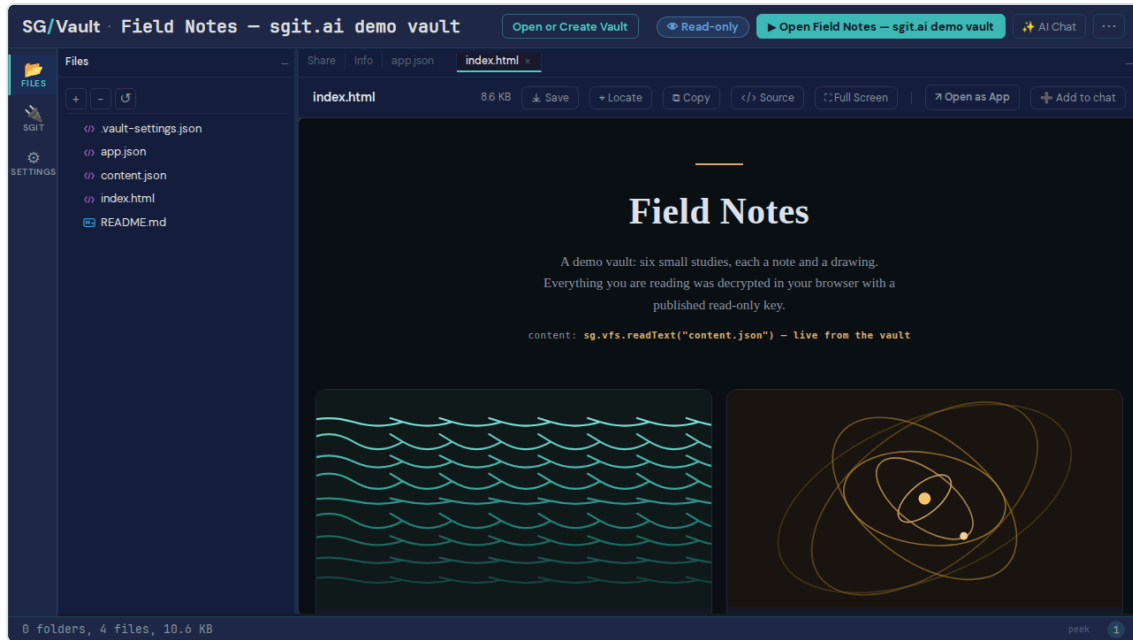
SCENE 11 OF 13 S11 9.0 S

Files, history, branches

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

HOW THIS IMAGE WAS CREATED

- ▶ Recorded a clip of the published vault “Field Notes”, opened live in SG/App
- ▶ Clicked “Open Vault”
- ▶ Used the last frame as the still



01:41

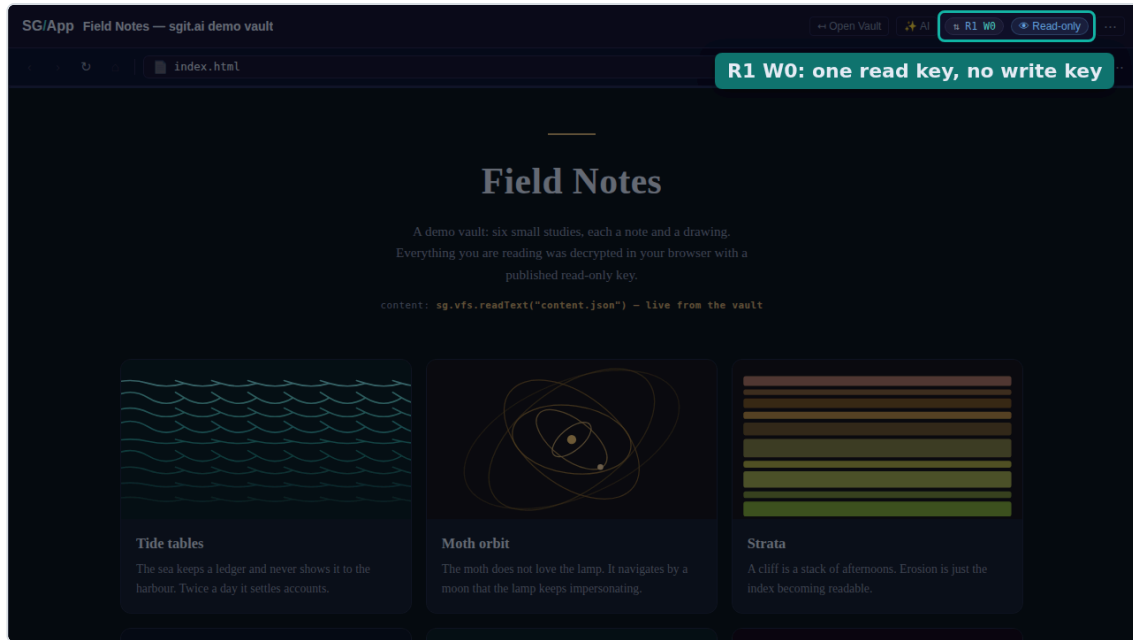
SCENE 12 OF 13 S12 5.3 S

Read-only by construction

And the badge says what the key allows. One read, zero writes.

HOW THIS IMAGE WAS CREATED

- ▶ Opened the published vault “Field Notes”, opened live in SG/App
- ▶ Spotlit the access badges in the header
- ▶ Added the label “R1 W0: one read key, no write key”



01:46

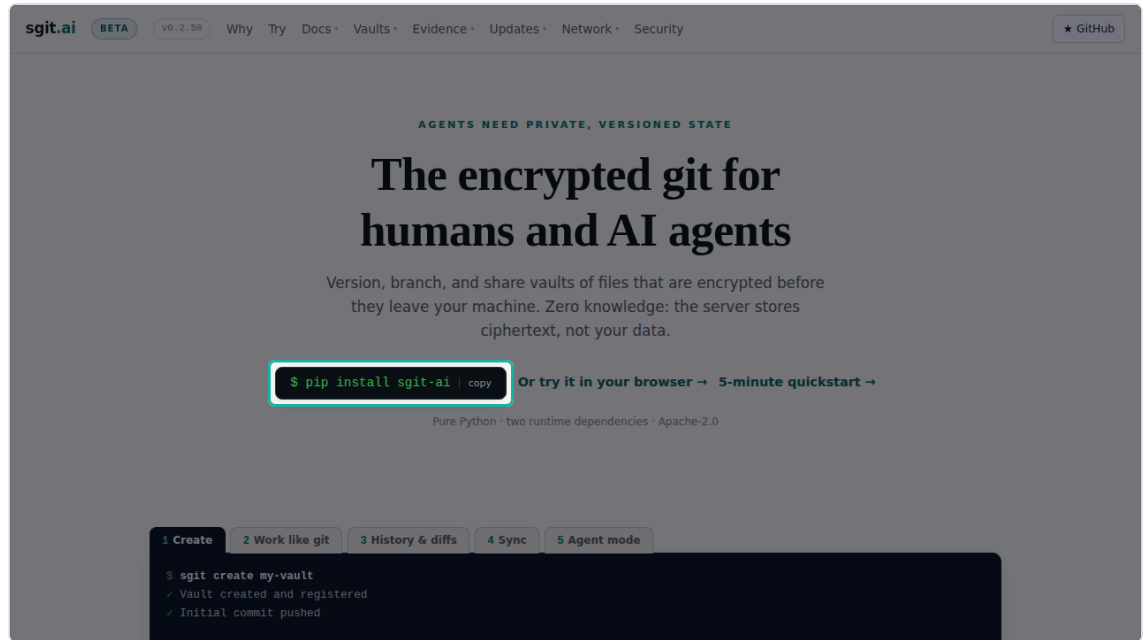
SCENE 13 OF 13 S13 7.3 S

pip install sgit-ai

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

HOW THIS IMAGE WAS CREATED

- ▶ Opened `https://sgit.ai/`
- ▶ Spotlighted the `pip install` line



How this video was made

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

MADE BY a Claude Code agent, unattended

SCRIPT 13 scenes + title and closing · 269 words · reel.json written first

NARRATION Kokoro-82M in the browser (2 workers), voice am_michael · 125.7 s of speech in 166 s · model load 7.6 s

PIPELINE WALL CLOCK 394 s: capture 93 + compose and narrate 176 + render 126

COMPUTE one 4-core container · agent session tokens not metered here

WHEN 2026-09-01

SCREENSHOTS 15 shot, 13 used · Playwright, headless · 93 s

RENDER video-creator, canvas + MediaRecorder, real time · 126 s

API COST \$0.00 · no LLM, TTS or image API calls

Transcript

The narration in order, as it is spoken. Copy it as plain text to post next to the video.

00:00

OPENING

In the next ninety seconds: what a vault is, what the server can and cannot see, and a published vault opened live in the browser.

00:09

SCENE 1 · ENCRYPTED BEFORE IT LEAVES

This is sgit. Git workflows over folders that are encrypted before anything leaves your machine.

00:17

SCENE 2 · THE SERVER SEES CIPHERTEXT

The server stores ciphertext under opaque identifiers. File names, contents and commit messages never reach it.

00:27

SCENE 3 · ONE COMMAND

Creating a vault is one command.

00:29

SCENE 4 · WORK LIKE GIT

Then you work like git. Status, commit, history, sync, and an agent mode built for machines.

00:37

SCENE 5 · SAME VAULT, IN THE BROWSER

SG Vault is the same vault in the browser. One encrypted wire format, no wrapper around the command line.

00:46

SCENE 6 · THE KEY NEVER LEAVES THE BROWSER

You open a vault by its key, carried in the URL fragment. The fragment is never sent, so the server never sees the key.

00:55

SCENE 7 · THREE PIECES, ONE WIRE FORMAT

Three pieces. The sgit command line, the SG Vault browser client, and SG Send, the transfer API that never sees plaintext.

01:08

SCENE 8 · THE APP IS THE VAULT

A vault can contain an app. Sharing the vault link is deploying the app.

01:14

SCENE 9 · READ KEYS, PUBLISHED ON PURPOSE

sgit dot ai publishes read keys for its demo vaults on purpose. A read key cannot become write access.

01:23

SCENE 10 · DECRYPTED IN YOUR BROWSER

Here is one, opened live. Everything on this page was decrypted in the browser with a published read only key.

01:32

SCENE 11 · FILES, HISTORY, BRANCHES

Behind the app is the vault itself. Files, history and branches, in the same file browser you get for any vault.

01:41

SCENE 12 · READ-ONLY BY CONSTRUCTION

And the badge says what the key allows. One read, zero writes.

01:46

SCENE 13 · PIP INSTALL SGIT-AI

pip install sgit dash ai. Or try it in your browser, at sgit dot ai.

01:54

CLOSING

This video was made by an agent, end to end, with no human at a screen and no API cost. The numbers are on the screen.

Renders

CUT	LENGTH	SIZE	SLIDES	TTS	RECORD	DRIFT	PIPELINE	API COST
landscape 1280×720 · Kokoro	02:05.75	9.4 MB	15	166 s	125.9 s	176 ms	302 s	\$0.00
shorts 1080×1920 · Kokoro	01:09.94	6.5 MB	8	96 s	70.0 s	127 ms	177 s	\$0.00
landscape 1280×720 · OpenRouter gpt-audio	01:50.55	8.0 MB	15	5 s	110.6 s	160 ms	127 s	\$0.147

Source of truth is reel.json. Stills in images/ (desktop) and images-short/ (phone); clips in clips/. Timecodes are the landscape cut's TTS durations. API cost is the narration provider's charge, priced per request from the OpenRouter generation endpoint; Kokoro runs locally at \$0.00.